

Final exam: HTML, CSS and JavaScript

Name _____

Date _____

Section	What it is	Marks	Conditions
A	Theory, 30 questions	30	45 minutes, on paper, no notes or devices
B	Practical project	70	Notes and MDN allowed. Deadline: _____

Read each question to the end before you answer. Most of them show you some code and ask what happens. Work it out in your head the way the browser would, one line at a time. The questions are not trying to trick you, but they do expect you to look carefully.

Section A: Theory

30 marks

Answer every question in the space given. Each question is worth 1 mark. When a question has more than one part, you can earn part of the mark.

HTML

1. Your page `index.html` sits in the main project folder. The logo is saved at `assets/images/logo.png`. This tag shows a broken image, and a screen reader has nothing to say about it:

```

```

Rewrite the tag so the image loads and can be described.

2. A shop page has two things you can click. "See all products" opens `products.html`. "Add to cart" changes a number on the same page. Which element should each one be, and why?

3. The headings on a page come in this order. What is wrong, and why does it matter to someone using a screen reader?

```
<h1>Our menu</h1>
<h3>Rice dishes</h3>
<h2>Soups</h2>
```

4. Clicking this link does nothing. Explain why, and fix it.

```
<a href="#prices">See prices</a>

<section class="prices">
  <h2>Prices</h2>
</section>
```

5. Your media queries are correct, but on a phone the page shows up tiny, like a desktop page zoomed out. Which line is missing from the <head>?

- (a) <meta charset="UTF-8">
- (b) <meta name="viewport" content="width=device-width, initial-scale=1">
- (c) <link rel="stylesheet" href="styles.css">
- (d) <html lang="en">

6. A classmate built a whole page out of <div> elements, and it looks perfect in the browser. Give two reasons to use <header>, <nav>, <main> and <footer> instead.

7. Name the best element, or pair of elements, for each of these.

- (a) A customer's review, quoted word for word on your page
- (b) A photo with a caption under it
- (c) The steps of a recipe, which must be done in order

8. This list has two mistakes. Find both.

```
<ul>
  <li>Jollof rice</li>
  <p>Fried rice</p>
  <li>Ofada rice <strong>(new)</li></strong>
</ul>
```

CSS

9. index.html is in the main project folder and the stylesheet is at css/styles.css. Which line links it correctly?

- (a) <link rel="stylesheet" href="/styles.css">
- (b) <link rel="stylesheet" href="css/styles.css">
- (c) <style src="css/styles.css"></style>
- (d) <script src="css/styles.css"></script>

10. Look at these rules and this paragraph.

```
p      { color: black; }
.note  { color: green; }
#intro { color: red; }
.note  { color: blue; }

<p id="intro" class="note">Doors open at 4pm.</p>
```

- (a) What colour is the text? (b) If you delete the #intro rule, what colour is it then, and why that one?

- 11.

```
.card {
  width: 300px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
```

- (a) With the default box-sizing, how wide is the card from the outside of its left border to the outside of its right border? (b) How wide is it if you add box-sizing: border-box?

12. Three cards sit on top of each other. You want them side by side, so you add `display: flex` to `.card`. Nothing changes. What is wrong?

```
<div class="cards">
  <div class="card">...</div>
  <div class="card">...</div>
  <div class="card">...</div>
</div>
```

13. On a phone 400px wide, are these cards in a row or in a column? Explain.

```
@media (max-width: 600px) {
  .cards { flex-direction: column; }
}

.cards { display: flex; flex-direction: row; }
```

14. An image with `transform: rotate(10deg)` turns around its centre. You want it to swing from the middle of its bottom edge instead, like a card fanned out in your hand. Which line do you add?

- (a) `rotate-from: bottom;`
- (b) `transform-origin: bottom center;`
- (c) `transform: rotate(10deg) bottom;`
- (d) `position: bottom center;`

15. A big decorative circle inside a section pokes out past the section's left and right edges, and now the whole page scrolls sideways on a phone. Which rule, on which element, cuts off the parts that stick out?

16. The root font size is 16px.

```
h1 { font-size: clamp(2rem, 6vw, 4rem); }
```

What size, in pixels, is the h1 when the browser window is (a) 400px wide, (b) 1000px wide, (c) 1400px wide?

17. Describe what you see when this rule is applied to a photo.

```
img {  
  mask-image: linear-gradient(to bottom, black 60%, transparent 100%);  
}
```

18. In the DevTools Styles panel, the color line in one of your rules has a line drawn through it. What is DevTools telling you, and what is a likely cause?

JAVASCRIPT

19. What does this print?

```
console.log("5" + 3, "5" - 3, typeof ("5" * "2"));
```

20. What happens when this runs, and how do you fix it?

```
const score = 10;  
score = score + 5;
```

21. Using these variables, write one `console.log` with a template literal that prints exactly `Tobi ordered 3 items for 4500 naira`. The 4500 must be worked out by your code, not typed in.

```
const name = "Tobi";  
const items = 3;  
const price = 1500;
```

22. Which two lines appear in the console, in order? Explain the second one.

```
function double(n) {  
  console.log(n * 2);  
}  
  
const result = double(4);  
console.log(result);
```

23. Write true or false for each: (a) `0 == ""` (b) `0 === ""` (c) `"7" === 7`. Then say which of `==` and `===` you should normally use, and why.

24. What does each call return?

```
function canEnter(age, hasTicket) {  
  if (age >= 18 && hasTicket) {  
    return "Welcome";  
  }  
  if (age >= 18 || hasTicket) {  
    return "Almost";  
  }  
  return "Sorry";  
}
```

(a) `canEnter(18, true)` (b) `canEnter(20, false)` (c) `canEnter(15, false)`

25. What does this print?

```
const prices = [1200, 800, 2500, 500];  
let total = 0;  
  
for (let i = 0; i < prices.length; i++) {  
  if (prices[i] >= 1000) {  
    total = total + prices[i];  
  }  
}  
  
console.log(total);
```

26. This prints three numbers, then stops with an error. Why, and what is the fix?

```
const names = ["Ada", "Bayo", "Chi"];

for (let i = 0; i <= names.length; i++) {
  console.log(names[i].length);
}
```

27. Write a loop that logs the name of every student who scored 50 or more. It should print Ada, then Zainab.

```
const students = [
  { name: "Ada", score: 72 },
  { name: "Emeka", score: 45 },
  { name: "Zainab", score: 88 }
];
```

28. The console shows Uncaught TypeError: Cannot read properties of null (reading 'addEventListener'). What is causing it here, and how do you fix it?

```
<button class="buy-btn">Buy now</button>

const button = document.querySelector("#buy-btn");
button.addEventListener("click", function () {
  // ...
});
```

29. This runs with no errors, but the message never turns red. Why?

```
/* styles.css */
.error { color: red; }

// script.js
message.classList.add(".error");
```

30. Whatever the user types, clicking Go always shows "Type your name first". Explain the bug and fix it.

```
<input id="name">
<button id="go">Go</button>
<p id="msg"></p>

const input = document.querySelector("#name");
const msg = document.querySelector("#msg");
const typed = input.value;

document.querySelector("#go").addEventListener("click", function () {
  if (typed === "") {
    msg.textContent = "Type your name first";
  } else {
    msg.textContent = `Welcome, ${typed}`;
  }
});
```

A ticket page for a one-day event

Make up a one-day event: a tech meetup, a gaming tournament, a small concert, a football viewing party, a class reunion, anything you like. Then build a one-page website for it. The page tells people what the event is and when things happen, and it has a booking panel where they pick tickets, see the price add up, and book.

No real payments happen and nothing is saved. Everything runs on the page, in the browser, with your own HTML, CSS and JavaScript. You can build all of it in a phone editor such as Acode or Spck, or on a laptop with VS Code.

Your project folder

```
event/  
  index.html  
  styles.css  
  script.js  
  notes.txt  
  images/
```

Part 1: HTML

14 marks

The page needs a correct skeleton: doctype, lang, charset, the viewport tag and a real title. Inside the body you need:

- a `<header>` with the event's name and a `<nav>` whose links jump to at least three sections on the page. Every link must land on its section.
- a `<main>` with at least four sections: About, Schedule, Tickets, and one more of your choice (FAQ, Speakers, Line-up or Venue).
- a schedule written as an ordered list with at least four items, each with a time.
- at least one image inside a `<figure>`, with real `alt` text and a `<figcaption>`.
- a `<footer>` with your name and the event date.
- exactly one `<h1>`, and no skipped heading levels.

The Tickets section holds the booking panel. It has three ticket types. Each one shows its name, its price, a minus button, the number chosen and a plus button. Under them goes the total, a name input, a phone number input with a label for each, a Book button, a Clear button and an empty paragraph for messages. Anything that does something on the same page is a `<button>`. Anything that goes somewhere is a link.

What is checked	Marks
Skeleton and head are complete and correct	2
Header, nav, main, sections and footer used for the right jobs	3
Every nav link lands on its section	2
Ordered schedule list, and a figure with real alt text and a caption	2
Booking panel complete, with inputs labelled and buttons used for actions	3
One h1, sensible heading order, every tag closed and nested correctly	2

Part 2: CSS

18 marks

All styling goes in `styles.css`. Do not use `style=""` attributes. Pick a small set of colours and one or two fonts and stick to them. Text must be easy to read against its background.

- Give the ticket types and other content blocks proper spacing with padding and margin, and set `box-sizing: border-box` on everything.
- Use Flexbox for the nav links and for the three ticket types, which sit side by side on a laptop.
- At 600px wide or less, the ticket types stack on top of each other. At 360px wide, nothing on the page makes it scroll sideways.
- Buttons and links change in some visible way on hover, and show a clear outline when you reach them with the keyboard (`:focus`).
- Use at least one technique from the CSS techniques session somewhere it makes sense: `clamp()` for the main heading, transform with `transform-origin`, a mask-image fade, a `filter: blur()` glow, or `overflow: hidden` to crop something on purpose.

What is checked	Marks
One linked stylesheet, no inline styles	1
Consistent colours and fonts, readable contrast	2
Spacing with padding and margin, <code>border-box</code> set	3
Flexbox for the nav and the ticket types	4
Stacks at 600px, no sideways scrolling at 360px	4
Hover and focus styles	1
A technique from the CSS techniques session, used with a reason	3

Part 3: JavaScript

28 marks

Start `script.js` with your tickets stored as an array of three objects. Change the names and prices if they do not fit your event.

```
const tickets = [
  { name: "Regular", price: 3000, count: 0 },
  { name: "VIP", price: 10000, count: 0 },
  { name: "Student", price: 1500, count: 0 }
];
```

The total must always be worked out from this array. If you change a price here, the total on the page must change with it.

The plus button adds one ticket of that type, and the minus button takes one away. A count can never go below 0. The whole order can have at most 10 tickets. If someone presses plus when they already have 10, the count stays the same and the message paragraph says "You can book at most 10 tickets." The numbers on the page update every time.

Write a function that loops through the array and *returns* the total price of the order. Use it to update the total on the page after every change. The total can show as plain digits, so "16500" is fine.

When the order has 5 or more tickets, take 10% off. Show the price before the discount, the price after it, and the words "Group discount applied". For example, 2 Regular and 3 Student is 5 tickets: ₦10,500 before the discount and ₦9,450 after it.

When the Book button is clicked, read the name and phone number at that moment and check them in this order:

1. If no tickets are chosen, show "Pick at least one ticket."
2. If the name is shorter than 2 characters, show "Enter your name."
3. If the phone number is not exactly 11 characters long, show "Enter an 11-digit phone number."

Show only the first problem found, and give the message paragraph the class error so it looks like a warning. If everything passes, remove that class, add the class success, and use a template literal to show a message like this:

```
Thanks, Ada. 5 tickets booked for ₦9450. We will text 08031234567.
```

The Clear button sets every count back to 0, empties both inputs and the message, and updates the page.

Finally, make one more part of the page respond to a click. For example, clicking an FAQ question shows or hides its answer, or clicking a speaker's name shows their short bio. Do it by adding and removing a class, and let your CSS do the hiding and showing.

What is checked	Marks
Tickets stored as an array of objects, and the total comes from it	3
Plus and minus work, never below 0, never above 10 in total, with the message	6
A function that loops and returns the total, shown after every change	6
Group discount from 5 tickets, both prices and the words shown	3
Book checks in the right order, reads the inputs on click, uses the error and success classes	6
Clear resets the counts, inputs, message and total	2
One more click interaction, done with a class	2

Part 4: Quality and your notes

10 marks

Keep your code tidy: indent it properly and give things names that say what they hold, such as `totalPrice` rather than `x`. Open the console and use every part of your page. No red errors should appear.

In `notes.txt`, answer these three questions in your own words. A few honest sentences each is enough.

1. Describe one bug you hit while building this. What did you see, how did you find the cause, and what fixed it?
2. Explain, line by line, what your `total` function does.
3. Which technique from the CSS techniques session did you use? Say what each property in it does on your page.

What is checked	Marks
No console errors during normal use	2
Tidy indentation and clear names	2
notes.txt, 2 marks per question	6

Rules

You may use your class notes, the workshop slides and MDN. You may ask an AI tool to explain an error message, but not to write your code. Your instructor may ask you to explain any part of your project. Marks for a part you cannot explain can be taken away.

Before you submit

- ☐ The page opens with no errors in the console.
- ☐ Every nav link lands on its section.
- ☐ You can add 10 tickets but not 11, and no count goes below 0.
- ☐ With the starting prices, 2 Regular and 3 Student shows ₦10,500 before the discount and ₦9,450 after it.
- ☐ Book with nothing chosen, with no name and with a short phone number, and you get the right message each time.
- ☐ Clear puts everything back to the start.
- ☐ At 360px wide, nothing scrolls sideways.
- ☐ `notes.txt` answers all three questions.

Zip the event folder, name the zip with your full name, for example `ada-okafor-exam.zip`, and submit it before the deadline.